# Product Requirements Document (PRD)

## Operational Analytics System — DBMS Project

## 1. PROJECT OVERVIEW

Build a full-stack operational analytics system that manages business operations including orders, users, products, and payments, and transforms that data into interactive dashboard insights. The system is built as a 3-tier architecture using React (frontend), Node.js + Express (backend), and MySQL (database). This is an academic DBMS project designed to demonstrate core database concepts in a working, demo-ready application.

## 2. TECH STACK

- **Frontend:** React, Recharts (for charts), Axios (for API calls), plain CSS or Tailwind
- **Backend:** Node.js + Express (REST APIs)
- **Database:** MySQL
- **Auth:** Hardcoded admin login (no JWT, no sessions — simple check in backend)
- **Real-time simulation:** Polling every 5 seconds (auto-refresh on dashboard)

## 3. DATABASE DESIGN

### 3.1 Tables

**Users**

- user_id (PK, AUTO_INCREMENT)
- name (VARCHAR)
- email (VARCHAR, UNIQUE)
- phone (VARCHAR)

- created_at (TIMESTAMP)

**Categories**

- category_id (PK, AUTO_INCREMENT)
- category_name (VARCHAR)

**Products**

- product_id (PK, AUTO_INCREMENT)
- name (VARCHAR)
- category_id (FK → Categories)
- price (DECIMAL)
- stock_quantity (INT)

**Orders**

- order_id (PK, AUTO_INCREMENT)
- user_id (FK → Users)
- order_date (TIMESTAMP)
- status (ENUM: pending, processing, shipped, delivered, cancelled)
- total_amount (DECIMAL)

**Order_Items**

- item_id (PK, AUTO_INCREMENT)
- order_id (FK → Orders)
- product_id (FK → Products)
- quantity (INT)
- unit_price (DECIMAL)

**Payments**

- payment_id (PK, AUTO_INCREMENT)
- order_id (FK → Orders)
- payment_date (TIMESTAMP)
- amount (DECIMAL)
- method (ENUM: credit_card, debit_card, upi, cash)

- status (ENUM: pending, completed, failed)

**Shipments**

- shipment_id (PK, AUTO_INCREMENT)
- order_id (FK → Orders)
- shipped_date (TIMESTAMP)
- delivery_date (TIMESTAMP)
- status (ENUM: pending, in_transit, delivered)
- tracking_number (VARCHAR)

**Audit_Logs**

- log_id (PK, AUTO_INCREMENT)
- action (VARCHAR — e.g., ORDER_PLACED, PAYMENT_DONE)
- table_name (VARCHAR)
- record_id (INT)
- changed_at (TIMESTAMP)
- description (TEXT)

## 3.2 Indexes

```
CREATE INDEX idx_orders_date ON Orders(order_date);
CREATE INDEX idx_orders_user ON Orders(user_id);
CREATE INDEX idx_payments_status ON Payments(status);
CREATE INDEX idx_products_category ON Products(category_i
d);
```

## 3.3 Views

**v_revenue_daily** — Groups total revenue by date from completed payments

**v_top_products** — Joins Order_Items and Products, sums quantity sold and revenue per product

**v_top_customers** — Joins Users and Orders, counts orders and sums spending per user

**v_customer_segments** — Uses CASE WHEN on order count and total spending to classify users as High-Value (order_count >= 10 AND total_spent >= 5000), Frequent (order_count >= 5), or Occasional (everything else)

**v_category_sales** — Joins Categories, Products, Order_Items to show revenue per category

**v_order_trends** — Groups order count by date to show order volume over time

### 3.4 Stored Procedures

**sp_place_order(user_id, product_id, quantity, payment_method)**

- Wraps everything in a transaction
- Checks stock_quantity before inserting — if stock is less than requested quantity, raise error: SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Out of stock'
- Inserts into Orders, Order_Items, Payments
- Deducts stock from Products
- On failure, rolls back entire transaction

**sp_monthly_report(month, year)**

- Returns total revenue, total orders, top 5 products, top 5 customers for the given month

### 3.5 Triggers

**trg_after_order_insert** — After INSERT on Orders, writes a row to Audit_Logs with action = 'ORDER_PLACED'

**trg_after_payment_update** — After UPDATE on Payments, writes a row to Audit_Logs with action = 'PAYMENT_UPDATED'

**trg_after_shipment_update** — After UPDATE on Shipments, writes a row to Audit_Logs with action = 'SHIPMENT_UPDATED'

---

## 4. BACKEND (Node.js + Express)

### 4.1 Structure

```
/backend
  server.js
  /routes
    auth.js
    orders.js
    products.js
    users.js
    analytics.js
    logs.js
  /db
    connection.js
```

## 4.2 API Endpoints

**Auth**

- POST /api/auth/login — checks if username === "admin" and password === "admin123", returns { success: true } or { success: false }

**Orders**

- GET /api/orders — fetch all orders with user name and status
- POST /api/orders — calls sp_place_order stored procedure
- PATCH /api/orders/:id/status — update order status

**Products**

- GET /api/products — fetch all products with category name and stock
- POST /api/products — add new product
- PATCH /api/products/:id/stock — update stock

**Users**

- GET /api/users — fetch all users
- POST /api/users — add new user

**Analytics**

- GET /api/analytics/revenue-daily — queries v_revenue_daily
- GET /api/analytics/top-products — queries v_top_products

- GET /api/analytics/top-customers — queries v_top_customers
- GET /api/analytics/segments — queries v_customer_segments
- GET /api/analytics/category-sales — queries v_category_sales
- GET /api/analytics/order-trends — queries v_order_trends
- GET /api/analytics/monthly-report?month=X&year=Y — calls sp_monthly_report

**Logs**

- GET /api/logs — fetches last 50 rows from Audit_Logs ordered by changed_at DESC

### 4.3 Database Connection

Use mysql2 package. Single connection pool in db/connection.js. All routes import and use this pool.

---

## 5. FRONTEND (React)

### 5.1 Structure

```
/frontend
  /src
    /pages
      Login.jsx
      Dashboard.jsx
      Orders.jsx
      Products.jsx
      Users.jsx
      Segments.jsx
      Logs.jsx
      MonthlyReport.jsx
    /components
      Navbar.jsx
      LineChart.jsx
      BarChart.jsx
      StatCard.jsx
      DataTable.jsx
```

```
    App.jsx
    api.js
```

## 5.2 Pages

**Login.jsx**

- Simple form with username and password fields
- On submit, calls POST /api/auth/login
- If success, stores a flag in localStorage (isLoggedIn = true) and redirects to Dashboard
- All other pages check this flag — if not logged in, redirect to Login

**Dashboard.jsx**

- Main analytics page
- Auto-refreshes all data every 5 seconds using setInterval
- Shows the following:
  - 4 stat cards: Total Revenue, Total Orders, Total Users, Total Products
  - Line chart: Revenue over time (from v_revenue_daily)
  - Line chart: Order trends over time (from v_order_trends)
  - Bar chart: Top 5 selling products (from v_top_products)
  - Bar chart: Category-wise sales (from v_category_sales)
- Has a date range filter that filters the revenue and order trend charts
- Has a category filter dropdown that filters the category sales bar chart

**Orders.jsx**

- Table showing all orders with columns: Order ID, Customer Name, Date, Status, Total Amount
- Button to place a new order — opens a simple form asking for User ID, Product ID, Quantity, Payment Method
- Calls POST /api/orders which triggers the stored procedure
- Dropdown per row to update order status

**Products.jsx**

- Table showing all products with columns: Product ID, Name, Category, Price, Stock
- Button to add a new product
- Inline stock update input per row

**Users.jsx**

- Table showing all users
- Button to add a new user

**Segments.jsx**

- Table showing customer segmentation from v_customer_segments
- Columns: User ID, Name, Total Orders, Total Spent, Segment (High-Value / Frequent / Occasional)
- Color-code each segment row — green for High-Value, yellow for Frequent, grey for Occasional

**Logs.jsx**

- Read-only table showing last 50 audit log entries
- Columns: Log ID, Action, Table Name, Record ID, Description, Timestamp
- Auto-refreshes every 10 seconds

**MonthlyReport.jsx**

- Two dropdowns: month and year selector
- On submit, calls GET /api/analytics/monthly-report
- Displays: total revenue, total orders, top 5 products table, top 5 customers table

## 5.3 Navigation

- Navbar with links to: Dashboard, Orders, Products, Users, Segments, Logs, Monthly Report
- Logout button that clears localStorage and redirects to Login

---

# 6. KEY DBMS CONCEPTS DEMONSTRATED

- **Normalization (3NF):** All tables are normalized, no redundant data, all non-key attributes depend only on the primary key
- **Indexes:** 4 indexes on frequently queried columns for query optimization
- **Views:** 6 views that power all dashboard analytics
- **Stored Procedures:** sp_place_order handles full order workflow with transaction and stock validation; sp_monthly_report generates monthly insights
- **Triggers:** 3 triggers that auto-populate Audit_Logs on order, payment, and shipment changes
- **Transactions:** sp_place_order wraps all inserts in BEGIN...COMMIT with ROLLBACK on failure
- **Constraints:** PK, FK, UNIQUE, ENUM, NOT NULL used throughout schema

---

## 7. DEMO FLOW (For Presentation)

1. Login with admin / admin123
2. Open Dashboard — show live charts
3. Go to Orders → Place a new order (fill the form)
4. Switch back to Dashboard — within 5 seconds charts update with new data
5. Go to Logs — show the audit entry auto-created by the trigger
6. Go to Segments — show customer classification
7. Go to Monthly Report — select current month, show generated report
8. Mention: "If stock runs out, the system blocks the order through a stored procedure with a transaction rollback"

---

## 8. SAMPLE DATA REQUIREMENTS

Seed the database with the following minimum data before demo:

- 10 users
- 4 categories (Electronics, Clothing, Food, Books)
- 15 products spread across categories
- 30 orders with varied dates over the past 3 months

- Corresponding order items, payments (mix of completed and pending), and shipments
- At least 3 users with enough orders to fall into High-Value or Frequent segment

---

## 9. CONSTRAINTS AND SIMPLIFICATIONS

- No real authentication — hardcoded admin login only
- No real-time WebSockets — polling every 5 seconds is sufficient for demo purposes and is described as "near real-time auto-refresh"
- Deployment is required — hosting provider and domain will be confirmed separately; the codebase must be built deployment-ready from the start (environment variables for DB credentials, no hardcoded localhost URLs, proper CORS configuration)
- No unit tests required
- Single admin user only — no multi-role system
- Frontend must be fully mobile responsive — all pages, charts, tables, and forms must work cleanly on mobile screen sizes; tables should be horizontally scrollable on small screens, charts should resize fluidly, and the navbar should collapse into a hamburger menu on mobile

---

## 10. SUCCESS CRITERIA

- All 8 database tables created and seeded with sample data
- All 6 views return correct data
- sp_place_order correctly handles stock check, transaction, and rollback
- sp_monthly_report returns correct aggregated data
- All 3 triggers correctly insert into Audit_Logs
- Dashboard charts load and auto-refresh
- All 7 pages are functional and navigable
- Demo flow in Section 7 runs without errors